



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo Práctico

Threading

25 de Octubre de 2024

Sistemas Operativos

Grupo: :(){: |: &}::

Integrante	LU	Correo electrónico
Iannantuono, Ignacio	1897/21	ignacio.iannantuono@gmail.com
Mercado, Pablo	1168/21	pablomercado0123@gmail.com
Weng, Xinyue	80/22	wengxinyue2@gmail.com



**Facultad de Ciencias Exactas y
Naturales**

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta
Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep.
Argentina

Tel/Fax: (+54 +11) 4576-3300

<http://www.exactas.uba.ar>

1 Introducción

En el presente trabajo práctico, implementamos y utilizamos un **HashMapConcurrente** en C++ para gestionar las compras de una página web. Más específicamente, nos enfocamos en el procesamiento concurrente de archivos de textos con las órdenes de compras, mediante el uso de threads. Por otro lado, analizamos que nuestras soluciones estén libres de condiciones de carrera, deadlocks e inanición.

La estructura de **HashMapConcurrente** implementa un diccionario o map, donde las claves representan los productos y los valores las apariciones en las órdenes de compras. Las claves son strings y los valores un número no negativo.

2 Lista atómica

Una lista atómica implementa operaciones atómicas, es decir, operaciones que una vez ejecutadas no pueden ser interrumpidas de manera que su ejecución se divida en varias partes. Esto permite que no se incurran en condiciones de carrera al momento de operar con la lista.

Nuestra tabla de hash maneja colisiones mediante listas enlazadas. Dada una implementación parcial de lista atómica, para almacenar los elementos en cada bucket, fue necesario implementar la operación **insertar**. Se aprovechó la función **compare_exchange_weak**, que es atómica, para comparar la **_cabeza** actual con el **head** viejo que se cargó. En caso de ser iguales, se asigna atómicamente **nuevoNodo** a **_cabeza**; en caso contrario, podría haberse dado el caso que tras asignarle el **_siguiente** al nodo nuevo que queríamos insertar (pero antes de actualizar **_cabeza**), otra instancia en paralelo de **insertar** haya completado una inserción, actualizando **_cabeza**. De esta forma, nuestro valor de **head** queda obsoleto, por lo que debemos actualizarlo hasta que coincidan para asignar **_cabeza** correctamente.

Dada una secuencia de llamados a insertar, los nodos de la lista podrían no quedar insertados exactamente en el mismo orden en el que las funciones fueron llamadas, debido a factores como el desalojo de threads. Sin embargo, esto no afecta a las funcionalidades provistas por **HashMapConcurrente**, ya que ninguna de sus funciones hace uso del orden de los nodos de una lista.

3 HashMapConcurrente

Las funciones implementadas para utilizar nuestra estructura **HashMapConcurrente** son **incrementar**, **claves**, **valor**, **promedio** y **promedioParalelo**.

3.1 incrementar()

En la función **incrementar**, podrían darse condiciones de carrera si dos threads (o más) intentan incrementar el valor de una misma clave, generando un valor inconsistente, diferente al resultado de realizar dos incrementos. Otro problema

podría generarse si dos threads no encuentran la misma clave, ya que se insertaría el mismo ítem más de una vez. Para lidiar con esto, consideramos como sección crítica el código que realiza la búsqueda de la clave y el incremento de su cantidad o la inserción del nuevo par, en caso de ser necesario. Implementamos la sección crítica para que pueda ser accedida por bucket, agregando un arreglo de mutexes que nos permite indexar por hash. Esto se debe a que cada bucket es independiente: si dos threads tienen claves de hash diferentes, realizan la operación de incrementar en dos listas diferentes. De esta manera, garantizamos que sólo haya contención de haber colisión de hash.

3.2 claves()

Para implementar **claves**, bloqueamos inicialmente toda la tabla (todos sus buckets) y, a medida que terminamos de copiar todas las claves de un bucket, desbloqueamos el acceso al mismo para no tener la tabla entera bloqueada durante todo el tiempo de ejecución. Una vez bloqueada la tabla, obtenemos un “snapshot” válido de la misma en un instante dado, evitando así condiciones de carrera.

3.3 valor()

La implementación de **valor** sigue una idea similar a **incrementar**. Bloqueamos sólo el bucket correspondiente para que, al momento de realizar la lectura, otro thread no pueda modificar el valor de la clave. Al igual que en **incrementar**, no nos interesa la información en los otros buckets y permitimos mayor concurrencia.

Esta función podría tener pequeñas inconsistencias temporales (las cuales son permitidas por el enunciado). Podría devolver un resultado que incluya actualizaciones del valor retornado posteriores al propio llamado de **valor**, como consecuencia de que al entrar en la función de **valor**, de alguna manera, se decide darle el lock a otra función que modifica ese mismo valor, la cual fue llamada posteriormente a **valor**.

También podría devolver un valor desactualizado, en caso de que, desde antes del llamado a **valor**, hubiese un llamado a **incrementar** para el mismo valor que nos interesa. Luego, al liberarse el mutex que esperaba, si el lock se lo queda la instancia de **valor**, entonces retornaremos una clave que no incluía la última actualización.

Por los patrones de sincronización que empleamos, las funciones implementadas hasta ahora no presentan deadlock. Observemos que tampoco hay inanición, ya que no bloqueamos toda la tabla durante el tiempo de ejecución de ninguna de ellas.

3.4 promedio() e incrementar()

Es necesario agregar primitivas de sincronización si queremos ejecutar **incrementar** y **promedio** concurrentemente. No es deseable que durante la ejecución de

`promedio` se modifique la tabla y, por lo tanto, se lean datos inválidos para su cálculo. Esencialmente, estamos frente al problema de lectores-escritores en un entorno concurrente.

Nuestra solución utiliza la estructura de sincronización del problema clásico lectores-escritores con prioridad para escritores, provista en Little Book of Semaphores¹. Le damos prioridad a los escritores ya que consideramos deseable que la información esté actualizada antes de ser leída. Esta prioridad implica que si venían llegando varios pedidos de lectura seguidos, y llega uno de escritura, los pedidos de lectura que lleguen tras este último, serán atendidos luego de que se complete la escritura. Además, si antes de completar esta escritura, llegasen más pedidos de escritura, estos serán atendidos antes que las lecturas, que estaban “en fila”. En una situación así, podría haber starvation de los lectores.

3.5 promedioParalelo()

La función `promedioParalelo` tiene la misma firma que `promedio`. Esta función obtiene el resultado mediante las sumas de los valores de cada fila y la división por la cantidad de ítems que hay en la tabla. Para esto, fue necesario tener como variables compartidas los acumuladores `suma` y `cantItems`, que cumplen tales roles y son pasados por referencia a `sumarEnBucket`. Al mismo tiempo, también agregamos la variable de sincronización `sumMutex` para evitar condiciones de carrera a la hora de actualizar ambos valores.

En la división del trabajo, consideramos que cada thread pueda “pedir” qué bucket leer. Al respecto, contamos con la variable atómica `bucketDisponible`, que indica el último bucket disponible para leer. De esta forma, nos aseguramos que cada thread pueda leer una fila a la vez y que, de haber más trabajo, tenga la posibilidad de leer otra fila disponible.

Por otro lado, para asegurar concurrencia con `incrementar`, incluimos en nuestro código las mismas primitivas de sincronización que `promedio`.

4 Carga de compras

Se proveen las funciones `cargarArchivo` y `cargarMultiplesArchivos`, que nos permitirán cargar las órdenes de compras en `HashMapConcurrente`.

`cargarArchivo` simplemente recorre el archivo y carga todas las palabras en la estructura de hash. Observamos que no fue necesario tomar un recaudo especial, ya que podíamos hacer uso de la función `incrementar`, que resuelve internamente la concurrencia.

La implementación `cargarMultiplesArchivos` es similar a `promedioParalelo`. Para que cada thread “pida” qué path leer, contamos con la variable compartida `filePathDisponible`, que es atómica. A diferencia de `promedioParalelo`, no necesitamos combinar los resultados de cada thread, simplemente utilizamos `cargarArchivo` para modificar la estructura del hash pasada por parámetro.

¹4.2.7 Writer-priority readers-writers solution: <https://greenteapress.com/semaphores.pdf#subsection.4.2.7>

5 Otros análisis

En una situación con un gran número de órdenes llegando constantemente, como en un Black Friday, podríamos tener tiempos lentos de respuesta para las lecturas, e incluso starvation. En especial, esto afectaría a las funciones `promedio` y `promedioParalelo` por los patrones de sincronización utilizados. Si en un Black Friday nos interesa más tomar pedidos que calcular el promedio, estaríamos satisfechos con nuestro sistema. Si sabemos a priori que las lecturas serán menos frecuente, podríamos mejorar nuestro sistema asignándole mayor prioridad a las lecturas, evitando que tanto estas y las escrituras tengan inanición. Para ello, podríamos cambiar las primitivas de sincronización en `promedio` y `promedioParalelo` y agregarlas en `claves`, en lugar de usar los mutexes. Esto transformaría a `claves` en una instancia más de lectura en el esquema de lectores-escritores.

Otra situación es que nuestro sistema cuente sólo con dos productos. En este caso, el sistema se podría simplificar bastante, ya no tendríamos la necesidad de utilizar una tabla de hash. Bastaría con tener 2 variables atómicas, una por cada producto. Se eliminaría también la necesidad de utilizar patrones complejos de sincronización. Además, ya que no usamos estructuras de locks, no tendríamos starvation y, al usar variables atómicas, no tendríamos race conditions. Por otra parte, creemos que usar threading ya no sería conveniente para calcular el promedio, ya que son sólo dos valores. Sin embargo, para procesar múltiples archivos de texto sí nos convendría tener una implementación que aproveche la concurrencia. s

5.1 calcularMedianaParalela

A continuación presentamos una idea sobre la implementación de una función que calcula la mediana de los valores de la tabla hash. Para calcular la mediana, requerimos ordenar todos los valores, pero antes tenemos que obtener todos los valores. Esto lo podemos hacer con una función similar a `claves`, pero que retorne las segundas componentes de los pares (clave, valor), así nos aseguramos la integridad de los datos al estar trabajando sobre una copia. Una vez que tenemos todos los valores en un vector “snapshot”, podríamos usar threading para paralelizar el cómputo. Podríamos dividir el array en k partes iguales, lo suficientemente chicas, y asignar un thread para que ordene cada una usando un algoritmo de sorting que sea eficiente para el tamaño de cada parte. Luego, podríamos combinar (merge) todos los k sub-arrays ordenados, también con la ayuda de los threads, mergeandolos de a pares en paralelo, hasta tener todo el vector “snapshot” ordenado. Luego, es solo cuestión de elegir el elemento medio según haya una cantidad par o impar de valores.

Si bien el uso de threads puede dar una mejora en performance, esto no viene sin un costo de overhead, por eso sería necesario algo de experimentación para encontrar una función que nos diga aproximadamente cuál es la cantidad óptima de pedazos (equivale a threads a utilizar) en los que dividir al vector para obtener la mejor relación costo beneficio, en función de la cantidad total de valores.

Uno de los problemas que podría tener esta solución es que si k es demasiado grande, a medida que vamos iniciando los últimos threads, los primeros podrían haber completado ya gran parte de su trabajo o incluso terminado, quedándose a la espera de que los demás terminen de ordenar. Para lidiar con esto, podríamos tener una estructura de datos común a todos los threads, que nos indique qué partes del array ya fueron ordenadas. Cuando un thread termina de ordenar, puede automáticamente empezar a mergear con alguna otra parte ordenada, sin tener que esperar a que todos terminen de ordenar. Esta estructura debería poder ser leída por 1 solo thread a la vez, para evitar race conditions, como el caso en que 2 threads que ya terminaron de ordenar traten de mergear con el mismo pedazo.

6 Conclusión

En este trabajo, logramos implementar las funciones de `HashMapConcurrente` aprovechando las herramientas dadas por el uso de threads. Principalmente, hicimos uso de patrones de sincronización y variables atómicas. Salvo en casos bordes mencionados, nuestras funciones cumplen con no presentar deadlocks, inanición y condiciones de carrera. Dadas estas condiciones, el sistema puede funcionar correctamente, manteniendo la integridad de los datos en un entorno concurrente.

En funciones como `claves` y `valor`, por los obstáculos generados por el scheduler, pueden darse situaciones en las que los datos retornados no estén actualizados. Nuestro trabajo considera que alcanza con retornar valores que provienen de una serialización válida de threads, por lo que, estos resultados son válidos. Otro problema, en relación al funcionamiento de los semáforos, es la inanición que se puede generar al querer acceder a un bucket; en el peor de los casos, por ejemplo, con muchos hilos de `incrementar`, no podemos ejecutar `valor`, ya que, de alguna manera, se decide nunca cederle el mutex. En general, bajo la asunción de que los semáforos realizan el signal correctamente, como no se bloquea toda la tabla durante el tiempo de ejecución de ninguna función, nuestro sistema no puede presentar starvation. Creemos que podemos evitar este tipo de problema y mejorar el tiempo de espera de los threads con otra implementación mejor de `ListaAtómica`, en la que tengamos métodos para obtener copias locales de listas, ya que podrían ser utilizadas en las implementaciones de `valor` y `claves`. De esta forma, se logra mayor concurrencia al no depender de los semáforos de cada bucket. Queda como trabajo a futuro implementar `HashMapConcurrente` con dicha `ListaAtómica`.